

Approfondissement dans l'exploitation de Docker - Partie 2

IV/ Déploiement de Portainer, interface graphique à Docker

2. Le chemin complet du dossier de l'hôte qui va permettre de conserver toutes les données de portainer sur l'espace disque de l'hôte est: **/var/lib/docker/volumes/portainer_data/_data**. Pour vérifier cela, j'ai tapé la commande suivante `docker volume inspect portainer_data`.

```
root@deb12:~# docker volume inspect portainer_data
[
  {
    "CreatedAt": "2025-03-06T08:52:03+01:00",
    "Driver": "local",
    "Labels": null,
    "Mountpoint": "/var/lib/docker/volumes/portainer_data/_data",
    "Name": "portainer_data",
    "Options": null,
    "Scope": "local"
  }
]
```

C'est le mount point.

4. L'option `-p 9443:9443` signifie que le conteneur va utiliser le port 9443, et que pour qu'on puisse accéder au conteneur depuis un autre appareil il faut utiliser le port 9443.
5. Pour accéder à l'application Web j'ai utilisé l'URL <https://10.50.0.17:9443/>.

Construire une image avec un Dockerfile et la publier sur Docker Hub - Partie 3

VIII/ Construction, publication et déploiement d'une nouvelle image en autonomie

1. Pour créer le contexte et le dockerfile, j'ai tapé les commandes suivantes:

```
root@deb12:~# mkdir ~/contexte/debian12-apache2
root@deb12:~# touch ~/contexte/debian12-apache2/dockerfile
```

2. Le contenu du dockerfile est:

```
FROM debian:12

LABEL Mainteneur="Mohamed [https://hub.docker.com/u/mohamedel0]"
LABEL description="Debian avec le serveur apache activé"

# Installation des logiciels
RUN apt update \
    && apt -y --no-install-recommends install \
        apache2 \
        tzdata \
        locales \
    && rm -rf /var/lib/apt/lists/* \
    && apt clean

# Configuration de la Timezone Europe/Paris et des locales
ENV LANG=fr_FR.UTF-8

RUN rm /etc/localtime \
    && ln -s /usr/share/zoneinfo/Europe/Paris /etc/localtime \
    && dpkg-reconfigure --frontend noninteractive tzdata \
    && sed -i 's/# fr_FR.UTF-8 UTF-8/fr_FR.UTF-8 UTF-8/' /etc/locale.gen \
    && echo 'LANG=${LANG}'>/etc/default/locale \
    && dpkg-reconfigure --frontend=noninteractive locales \
    && update-locale LANG=${LANG}

# Modifier le fichier html par défaut
RUN echo "<h1>Le serveur apache est fonctionnel</h1>" > /var/www/html/index.html

EXPOSE 80

CMD ["/usr/sbin/apache2ctl", "-D", "FOREGROUND"]
```

3. La commande qui permet de créer l'image est:

```
root@deb12:~# docker build -t mohamedel0/debian12-apache2:1.0 ~/contexte/debian12-apache2
```

4. La commande pour lancer le conteneur est:

```
root@deb12:~# docker run --name apache2 -d -p 10.50.0.17:8008:80 mohamedel0/debian12-apache2:1.0
```

5.



6. Pour publier l'image sur le Docker Hub, j'ai tapé les commandes suivantes:

```
docker push mohamedel0/debian12-apache2:1.0
```



mohamedel0/debian12-apache2

By [mohamedel0](#) · Updated 1 minute ago

IMAGE

☆0 ↓0

Overview **Tags**

Sort by

Newest ▾

🔍 Filter tags

TAG

■ [1.0](#)

Last pushed 1 minute by [mohamedel0](#)

Digest	OS/ARCH	Last pull
cbb738821b21	linux/amd64	less than 1 day

Déploiement de la distribution Kali-Linux - Partie 4

III/ Personnalisation de l'image

2. J'ai remplacé les deux lignes par:

```
ENV LANG=fr_FR.UTF-8
RUN rm /etc/localtime \
    && ln -s /usr/share/zoneinfo/Europe/Paris /etc/localtime \
    && dpkg-reconfigure --frontend noninteractive tzdata \
    && sed -i 's/# fr_FR.UTF-8 UTF-8/fr_FR.UTF-8 UTF-8/' /etc/locale.gen \
    && echo 'LANG=${LANG}'>/etc/default/locale \
    && dpkg-reconfigure --frontend=noninteractive locales \
    && update-locale LANG=${LANG}
```

4.

```
GNU nano 7.2          env *
# !/bin/bash

# RDP port
XRDP_PORT=13389
# VNC ports
# XVNC_DISPLAY=8
# XVNC_PORT=5908
# SSH port
XSSH_PORT=20022

# Container desktop environment
# Options: "xfce" "mate" "kde" "e17" "gnome" "i3" "i3-gaps" "live" "lxde"
XDESKTOP_ENVIRONMENT="xfce"

# Remote access method for desktop
# Options: "vnc" "rdp" "x2go"
# Once rdp is up, it may take a moment to load
# Note: Only xfce works with rdp right now
XREMOTE_ACCESS="rdp"

# Kali packages
# Options: "arm" "core" "default" "everything" "firmware" "headless" "labs" "large" "nethunter"
XKALI_PKG="core"

# Network configuration
# Options: "bridge" "host"
XNETWORK="bridge"

# Target build architecture
# Options: "amd64" "arm64"
# Other untested options: https://docs.docker.com/build/building/multi-platform/
XBUILD_PLATFORM="amd64"

# Name for local custom Docker image
DOCKERIMG="mohamedel0/kalilinux:core"
# Name for Docker container generated from custom image
CONTAINER="kali-linux-core"
# Host directory to mount to container
HOSTDIR="/root/kali-linux-core"
# Container directory to mount host directory
CONTAINERDIR="/home/userkali"

# Username for container user
USERNAME="userkali"
# Password for container user
PASSWORD="root"

⌘ Aide      ⌘ Écrire   ⌘ Chercher  ⌘ Couper    ⌘ Exécuter  ⌘ Emplacement  ⌘ Annuler  ⌘ Marquer  ⌘ -> Crochet  ⌘ Précédent
⌘ Quitter   ⌘ Lire fich. ⌘ Remplacer ⌘ Coller    ⌘ Justifier ⌘ Aller ligne  ⌘ Refaire  ⌘ Copier    ⌘ Retrouver  ⌘ Suivant
```

6. Suppression de l'exposition du port VNC dans le Dockerfile:

```
ARG DESKTOP_ENVIRONMENT
ARG REMOTE_ACCESS
ARG KALI_PACKAGE
ARG SSH_PORT
ARG RDP_PORT
# ARG VNC_PORT
# ARG VNC_DISPLAY
ARG BUILD_ENV
ARG HOSTDIR
ARG CONTAINERDIR
ARG UNAME
ARG UPASS
```

```
EXPOSE ${SSH_PORT} ${RDP_PORT}
      # ${VNC_PORT}
WORKDIR "/root"
ENTRYPOINT ["/bin/bash"]
CMD ["/startkali.sh"]
```

8.

```
#if [ -z $XVNC_DISPLAY ];
#then
#   XVNC_DISPLAY=8
#fi

#if [ -z $XVNC_PORT ];
#then
#   XVNC_PORT=590$XVNC_DISPLAY
#fi
```

```

docker image build --platform linux/$XBUILD_PLATFORM \
    -t $DOCKERIMG \
    --build-arg DESKTOP_ENVIRONMENT=$XDESKTOP_ENVIRONMENT \
    --build-arg REMOTE_ACCESS=$XREMOTE_ACCESS \
    --build-arg KALI_PACKAGE=$XKALI_PKG \
    --build-arg RDP_PORT=$XRDP_PORT \
    --build-arg SSH_PORT=$XSSH_PORT \
    --build-arg BUILD_ENV=$XBUILD_PLATFORM \
    --build-arg HOSTDIR \
    --build-arg CONTAINERDIR \
    --build-arg UNAME=$USERNAME \
    --build-arg UPASS=$PASSWORD \
    .

# #####
# create the container
# #####
# call docker create and pass on all
# the choices for network and ports that the
# user has made in the menu
# #####

docker create --name $CONTAINER \
    --network $XNETWORK \
    --platform linux/$XBUILD_PLATFORM \
    -p $XRDP_PORT:$XRDP_PORT \
    -p $XSSH_PORT:$XSSH_PORT \
    -t \
    -v $HOSTDIR:$CONTAINERDIR \
    $DOCKERIMG

```

J'ai supprimé le VNC_PORT et le VNC_DISPLAY de docker image et docker create.

10. Pour créer le \$HOSTDIR j'ai tapé la commande suivante: `mkdir /root/contexte/kali-linux-docker/kali-linux-core`

```

# Host directory to mount to container
HOSTDIR="/root/contexte/kali-linux-docker/kali-linux-core"

```

IV/ Création d'une première image et déploiement du conteneur

3. La commande de création du conteneur en remplaçant chaque variable par sa valeur est:

```

docker create --name kali-linux-core \
    --network bridge \
    --platform linux/amd64 \
    -p 13389:13389 \
    -p 20022:20022 \
    -t \
    -v /root/contexte/kali-linux-docker/kali-linux-core:/home/userkali \
    mohamedel0/kalilinux:core

```

4. La commande de démarrage du conteneur est: `docker start kali-linux-core`

5. La commande qui permettra d'accéder à la distribution en SSH est: `ssh userkali@10.50.0.43 -p 20022`

```
OpenSSH SSH client
Microsoft Windows [version 10.0.19044.1706]
(c) Microsoft Corporation. Tous droits réservés.

P:\>ssh userkali@10.50.0.43 -p 20022
userkali@10.50.0.43's password:
Linux a8043b724ec2 6.1.0-29-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.1.123-1 (2025-01-02) x86_64

The programs included with the Kali GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

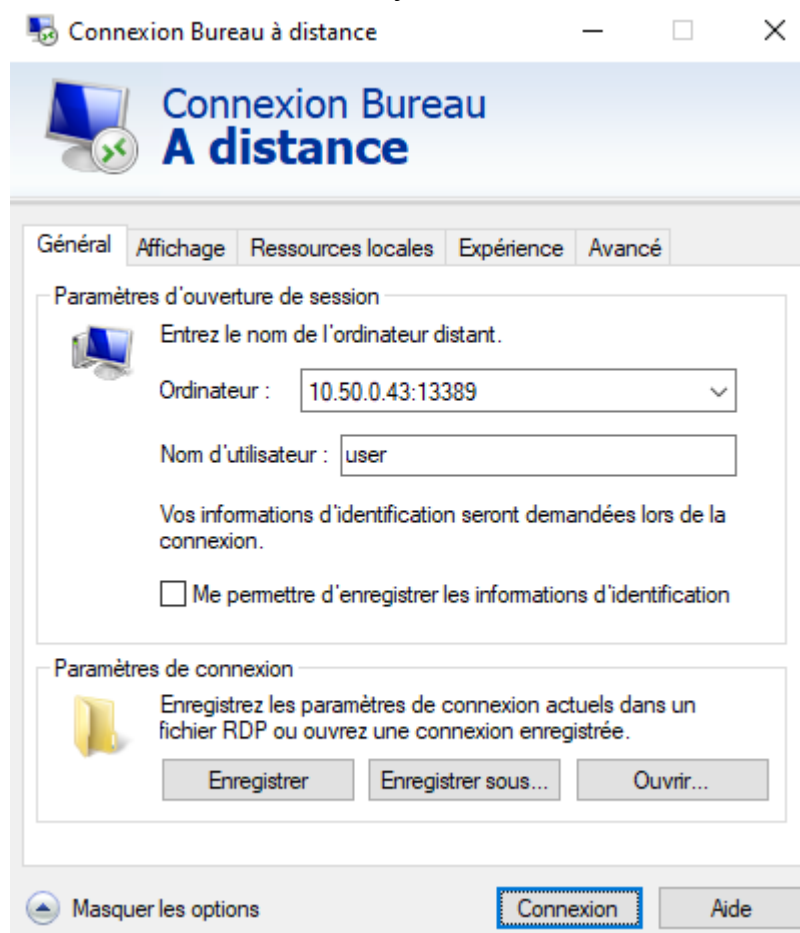
Kali GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Thu Mar 20 09:09:01 2025 from 10.54.3.1
(Message from Kali developers)

This is a minimal installation of Kali Linux, you likely
want to install supplementary tools. Learn how:
@ https://www.kali.org/docs/troubleshooting/common-minimum-setup/

(Run: "touch ~/.hushlogin" to hide this message)
userkali@a8043b724ec2:~$
```

8. Pour configurer le client bureau à distance pour accéder à l'interface graphique de Kali, j'ai tapé la commande suivante: `useradd --create-home --shell /bin/bash user && echo "user:pass123" | chpasswd`.

Pour me connecter à kali-linux, j'ai exécuté l'outil `mstsc` sur mon PC physique:



Le résultat de la connexion est:



9.

[Repositories](#) / [kalilinux](#) / [General](#)

<

mohamedel0/kalilinux

Last pushed 3 minutes ago

Add a description

Add a category

General

Tags

Image Management BETA

Collaborators

Webhooks

Settings

Tags

This repository contains 1 tag(s).

Tag	OS	Type	Pulled	Pushed
core		Image	less than 1 day	3 minutes

[See all](#)

10. La taille de mon image est de 2.03 Go.

```
root@deb12:~/contexte/kali-linux-docker# docker images
REPOSITORY          TAG         IMAGE ID      CREATED        SIZE
mohamedel0/kalilinux core        0af17e4ecbb7  2 hours ago   2.03GB
```

V/ Optimisation du déploiement

V/1 Gestion des volumes

3. Pour associer le volume, on modifie cette commande dans le fichier *build*:

```
docker create --name $CONTAINER \  
  --network $XNETWORK \  
  --platform linux/$XBUILD_PLATFORM \  
  -p $XRDP_PORT:$XRDP_PORT \  
  -p $XSSH_PORT:$XSSH_PORT \  
  -t \  
  -v home_$CONTAINER:$CONTAINERDIR \  
  $DOCKERIMG$
```

Déploiement du service phpIPAM avec Docker Compose - Partie 5

Préparation de docker compose

Création et modification du fichier compose.yaml

Tout d'abord il faut créer un dossier *dc_phpipam* dans le dossier contexte. Dans ce dossier on va créer un fichier nommé **compose.yaml*. Avec la commande `nano compose.yaml`, on va ajouter le contenu suivant au fichier:

```
GNU nano 7.2                                     compose.yaml
services:
  phpipam-web:
    image: phpipam/phpipam-www:latest
    container_name: phpipam-web
    ports:
      - "8099:80"
      - "44399:443"
    environment:
      - TZ=Europe/Paris
      - IPAM_DATABASE_HOST=phpipam-mariadb
      - IPAM_DATABASE_PASS=mysql&Ipam0
      - IPAM_DATABASE_WEBHOST=%
    restart: unless-stopped
    volumes:
      - phpipam-logo:/phpipam/css/images/logo
      - phpipam-ca:/usr/local/share/ca-certificates:ro
    depends_on:
      - phpipam-mariadb

  phpipam-cron:
    image: phpipam/phpipam-cron:latest
    container_name: phpipam-cron
    environment:
      - TZ=Europe/Paris
      - IPAM_DATABASE_HOST=phpipam-mariadb
      - IPAM_DATABASE_PASS=mysql&Ipam0
      - SCAN_INTERVAL=1h
    restart: unless-stopped
    volumes:
      - phpipam-ca:/usr/local/share/ca-certificates:ro
    depends_on:
      - phpipam-mariadb

  phpipam-mariadb:
    image: mariadb:latest
    container_name: phpipam-db
    environment:
      - MYSQL_ROOT_PASSWORD=mysql&Root0
    restart: unless-stopped
    volumes:
      - phpipam-db-data:/var/lib/mysql

volumes:
  phpipam-db-data:
  phpipam-logo:
  phpipam-ca:
```

[Lecture de 46 lignes]

^G Aide ^O Écrire ^W Chercher ^K Couper ^T Exécuter ^C Emplacemer
^X Quitter ^R Lire fich. ^M Remplacer ^U Coller ^J Justifier ^_ Aller ligr

Pour lancer les conteneurs à partir du fichier compose.yaml, taper la commande `docker compose up -d`.

```
root@deb12:~/contexte/dc_phpipam# docker compose up -d
[+] Running 16/16
  ⬇️ phpipam-cron Pulled
  ⬇️ 4f4fb700ef54 Pull complete
  ⬇️ phpipam-mariadb Pulled
  ⬇️ 2726e237d1a3 Pull complete
  ⬇️ 0b86886c6aaa Pull complete
  ⬇️ 2b221cf763a8 Pull complete
  ⬇️ 5e4180757702 Pull complete
  ⬇️ 43028b9f5f8e Pull complete
  ⬇️ bbef7eafa75b Pull complete
  ⬇️ ab732728101f Pull complete
  ⬇️ 0c9f57c1bb30 Pull complete
  ⬇️ phpipam-web Pulled
  ⬇️ 0a9a5dfd008f Pull complete
  ⬇️ 6a62d74e8ef1 Pull complete
  ⬇️ 249b3c2b26c5 Pull complete
  ⬇️ 09e2abc83335 Pull complete
[+] Running 7/7
  ⬇️ Network dc_phpipam_default Created
  ⬇️ Volume "dc_phpipam_phpipam-logo" Created
  ⬇️ Volume "dc_phpipam_phpipam-ca" Created
  ⬇️ Volume "dc_phpipam_phpipam-db-data" Created
  ⬇️ Container phpipam-db Started
  ⬇️ Container phpipam-cron Started
  ⬇️ Container phpipam-web Started
root@deb12:~/contexte/dc_phpipam#
```

Normalement les conteneurs ont bien été lancés, mais on peut le vérifier avec la commande `docker ps`.

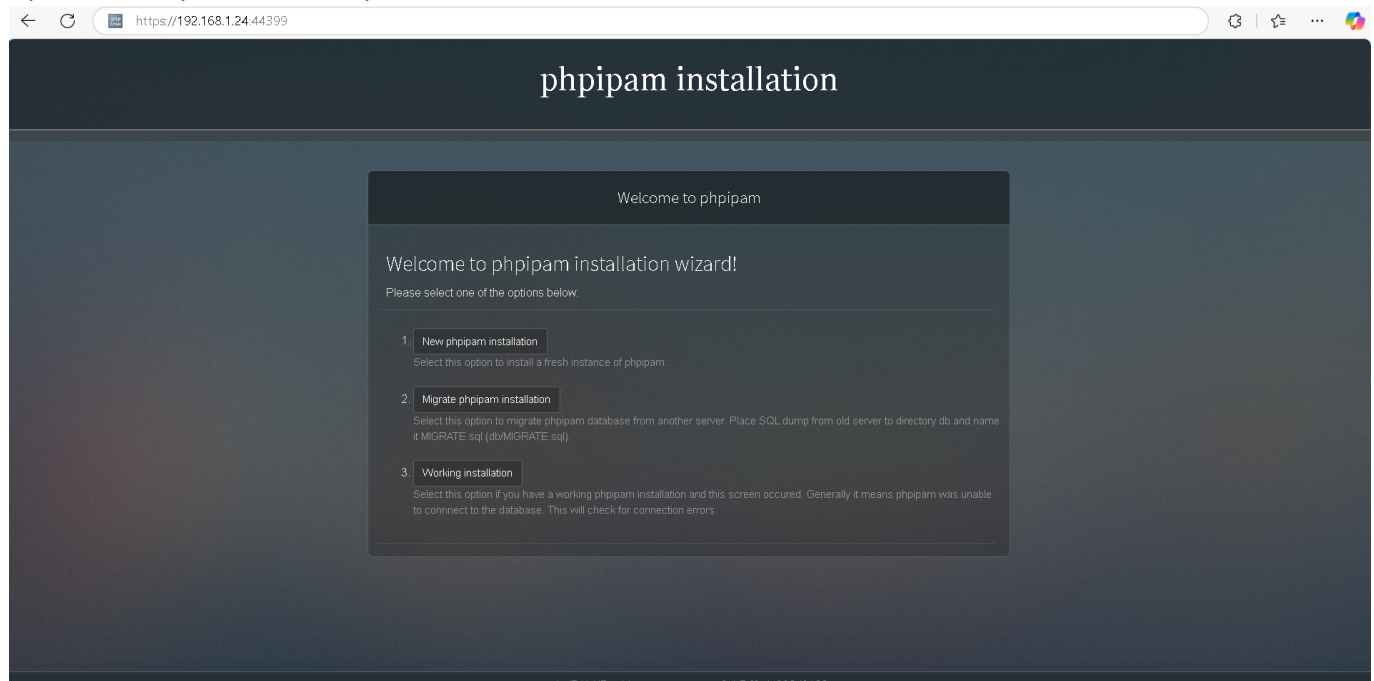
Accès à phpipam

Dans mon cas, pour accéder en HTTPS à l'application phpIPAM, je dois taper l'URL suivante:

<https://192.168.1.24:44399>

Dernières configurations de phpipam

Après avoir tapé l'URL correspondante, on a ce résultat:



Il faut suivre ensuite les étapes suivantes:

- Cliquer sur *New phpIPAM installation*.
- Cliquer sur *Automatic database installation*.
- Remplir les valeurs demandées.
- Puis cliquer sur *Install phpipam database*.
- Cliquer sur *Continue*.
- Remplir les valeurs demandées.
- Cliquer sur *Save settings*.

On peut maintenant se connecter avec le compte administrateur qu'on a créé.

Déploiement de l'application NetworkManager

Il faut créer un fichier dockerfile et écrire le texte suivant:

```
FROM debian:latest

LABEL Mainteneur="Mohamed El Mesbahi"
LABEL description="Debian avec le NetworkManager"

RUN apt update \
    && apt-y--no-install-recommends install \
        apache2 \
        ca-certificates ssl-cert \
        composer \
        apt-transport-https gpg \
```

```

php \
php-xml php-curl php-twi \g
php-twig-i18n-extension \php-zip
php-tcpdf \
php-webmozart-assert php \-symfony-yaml
php-symfony-var-exporter \
php8.2-bz2 php8.2-gd \
php8.2-mysql php8.2-zip \
unzip \
git \
mariadb-client \
vim \
nano sudo \
net-tools dnsutils \
wget \
curl \
nmap \
tcpdump \
openssh-server \
locales \
&& rm-rf /var/lib/apt/lists/* \
&& apt clean

```

```
ENV LANG=fr_FR.UTF-8
```

```

RUN rm /etc/localtime \
&& ln-s /usr/share/zoneinfo/Europe/Paris /etc/localtime \
&& dpkg-reconfigure--frontend noninteractive tzdata \
&& sed-i 's/# fr_FR.UTF-8 UTF-8/fr_FR.UTF-8 UTF-8/' /etc/locale.gen \
&& echo 'LANG=${LANG}'>/etc/default/locale \
&& dpkg-reconfigure--frontend=noninteractive locales \
&& update-locale LANG=${LANG}

```

```

RUN chmod a=rwx /var/www/html/NetworkManager
RUN wget-O composer-setup.php https://getcomposer.org/installer
RUN php composer-setup.php--install-dir=/usr/local/bin--filename=composer
RUN composer self-update
RUN cd /var/www/html
RUN git clone https://forge.apps.education.fr/reseau-
certa/applications/networkmanager.git
RUN chown-R www-data:www-data NetworkManager
RUN cd NetworkManager
RUN composer install

```

```

RUN echo "PP_NAME=NetworkManager
APP_ENV=local
APP_KEY=base64:6lBl6mDUsx1F34f6V0Imc6KI2jZYi67GVhqW6VffmhE=
APP_DEBUG=true
WWWUSER=root
WWWGROUP=root
LOG_CHANNEL=stack
LOG_LEVEL=debug
DB_CONNECTION=mysql

```

```
DB_HOST=192.168.1.24
DB_PORT=3306
DB_DATABASE=network_manager
DB_USERNAME=netuser
DB_PASSWORD=root
BIND_BASE_URL=http://apibind:8000
BIND_API_KEY=tfgz0FUNHop4JRtLw2st/YyFavu3UMow8vsadVC7t44f5EkVQ4o4IsDZHX0/MLHg
BROADCAST_DRIVER=log
CACHE_DRIVER=file
QUEUE_CONNECTION=sync
SESSION_DRIVER=file
SESSION_LIFETIME=120
MEMCACHED_HOST=memcached
RUN cp default-ssl.conf /etc/apache2/sites-available/default-ssl.conf

RUN rewrite ssl
RUN default-ssl

RUN useradd--create-home--shell /bin/bash userssh && echo "userssh:mdpuser" |
chpasswd

RUN service ssh start
RUN service apache2 start

EXPOSE 22
EXPOSE 80
EXPOSE 443

CMD ["/usr/sbin/sshd","-D"]
CMD ["/usr/sbin/apache2ctl","-D","FOREGROUND"]
```